

Aula 05 – Funções em Python 3

Parte 1 – Revisão Teórica

Blocos de Instrução

Em Python blocos de instrução são definidos pela **indentação**, diferente da maioria das linguagens que usam as chaves **{ }** para marcar os blocos de instrução.

Assim Python requer uma **indentação** padronizada, se em outras linguagens como C ou Java a **indentação** é utilizada somente para **melhor visualização**.

Python ela define onde começa e onde termina um bloco de instruções.

Esses blocos de instruções em Python são chamados de **suites**.

O que são as PEP do Python: **PEP** significa **P**ython **E**nhancement **P**roposal.

Um PEP é um documento de design que fornece informações para a comunidade Python ou descreve um novo recurso para Python ou seus processos ou ambiente.

O PEP deve fornecer uma especificação técnica concisa do recurso e uma justificativa para o recurso.
[\[http://legacy.python.org/dev/peps/pep-0001/#what-is-a-pep\]](http://legacy.python.org/dev/peps/pep-0001/#what-is-a-pep)

Aqui vamos chamar a atenção para a **PEP 8** que é o guia de estilos para códigos em Python, a PEP 8 recomenda o uso de **4 espaços por nível de indentação**, atenção a PEP 8 em bem clara quanto a preferência pelo uso de espaços no lugar da tabulação.

É recomendável o uso dos estilos com base na PEP 8, mas, você pode usar o estilo que desejar desde que a indentação tenha um tamanho padrão para separar os blocos de instrução e de preferência esse padrão seja seguido no código inteiro.

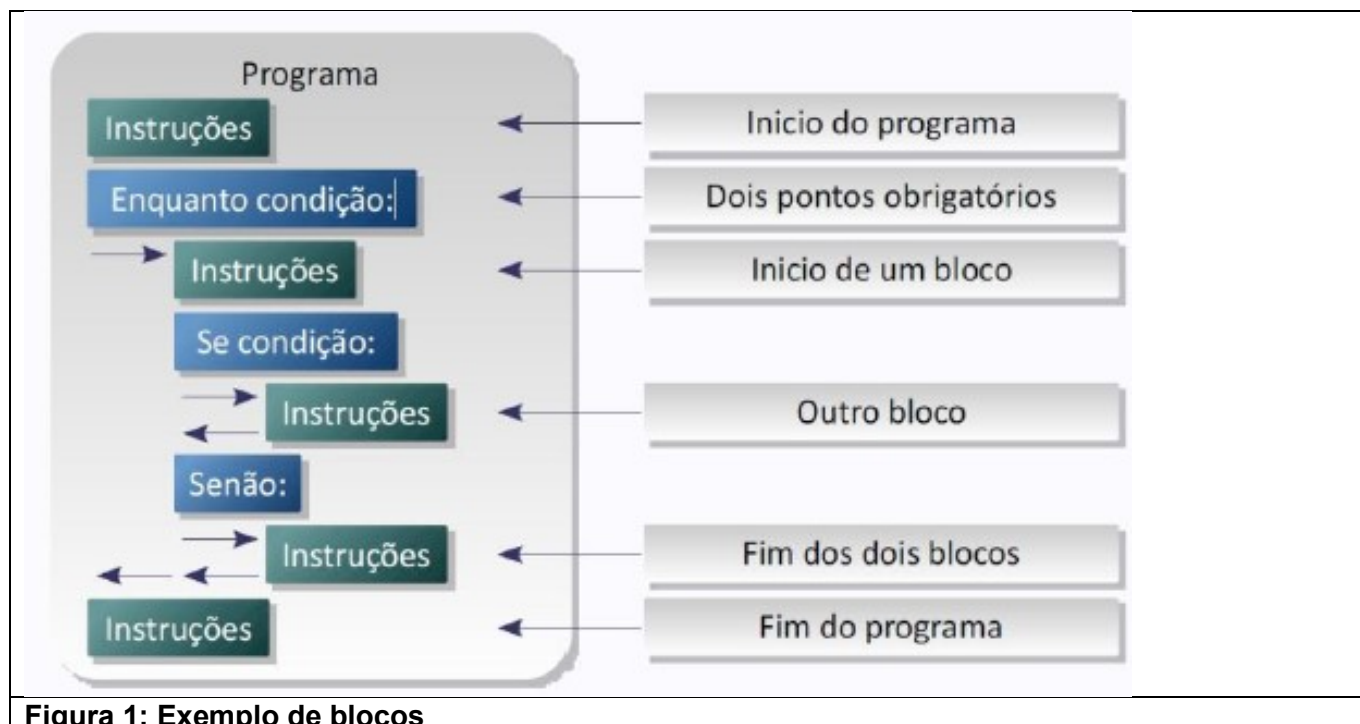


Figura 1: Exemplo de blocos

Resumindo sobre blocos de instrução

1. Os blocos de instrução são construídos por nível de indentação.
2. A indentação deve se manter constante por todo o código.
3. Para a indentação é recomendado o uso de **4 espaços**.

INTRODUÇÃO A FUNÇÃO NO PYTHON

Funções são blocos de instrução que podem ser invocados de qualquer parte do nosso código. Toda função, por definição:

- a) Possui um nome,
- b) Pode receber parâmetros e
- c) Pode retornar valores.

Nem toda função receberá parâmetros, da mesma forma, que nem toda função retornará valores. Tudo dependerá da situação, ainda assim, a estrutura sempre seguirá um padrão e por isso, é facilmente reconhecido.

A ideia mais primitiva das funções é o agrupamento de instruções, proporcionando um meio simples de execução sequencial e que pode ser invocado de qualquer parte a qualquer momento. Dessa forma, toda função irá possuir um bloco de instrução e quando invocada, o cursor de execução do programa se posicionará na primeira linha desse bloco e a execução ocorrerá de cima para baixo, uma instrução após outra.

A função pode definir em seu cabeçalho um conjunto de valores que devem ser enviados quando a função for invocada. Desta forma, a função só será invocada se os valores forem também definidos. Após a execução das instruções definidas no bloco de instrução da função, a mesma poderá retornar um valor e, dessa forma, temos um ciclo onde os dados são enviados, processados e retornados. Por isso é que raramente devemos utilizar variáveis globais, até porque, todos valores podem ser enviados e retornados pelas funções de maneira explícita.

Como todos os parâmetros estão por definição associados a função e possuem seu escopo restrito a esta, acabamos por ter uma situação em que é fácil depurar e encontrar problemas, até porque, é de certeza absoluta que o valor de um parâmetro foi alterado no escopo da função - situação que não ocorre quando os valores são manipulados por variáveis globais, até porque, estas podem ter seus valores alterados em qualquer lugar por qualquer função.

Uma função é um bloco de instrução independente (**sub-rotina ou miniprograma**), até porque, possui escopo restrito, e após a execução das instruções definidas em seu bloco de instrução, ela é finalizada, as variáveis desalocadas e o cursor de execução do programa volta ao ponto onde a função foi invocada.

DEFINIÇÃO

A sintaxe de uma função é definida por 3 partes:

- a) Nome,
- b) Parâmetros e
- c) Corpo, o qual agrupa uma sequência de linhas que representa algum comportamento

Qual a função da função?

- Evita repetição de código
- Deixa o código Menor
- Mais legível
- Mais modularizado

Tipos

- a) Função Interna (builtin) ou embuidas
- b) Importando funções externas prontas e disponíveis
- c) Função personalizada (definida pelo programador)

a) Funções internas (embutidas) ou builtin no Python

Funções builtin no Python

A biblioteca do Python contém vários componentes embutidos, que podem ser utilizados em qualquer parte do código sem a necessidade de um import. Um exemplo disso é a função **max()**, que retorna o maior elemento de uma lista que lhe é passada por parâmetro.

No código abaixo, temos alguns exemplos de uso:

```
print("Funções internas - builtin ou embutidas")
maior_numero = max(5.5, 5.6, 5.7, 8.3)
menor_numero = min(3, 6, 7, 10)
maior_letra = max('a', 'b', 'c')
menor_letra = min('a', 'b', 'c')

print("O Nro maximo da lista =", maior_numero)
print("O Nro minimo da lista =", menor_numero)
print("A maior letra da lista =", maior_letra)
print("A menor letra da lista =", menor_letra)

nome = "henrique"
nome.lower()
nome.upper()
nome.capitalize()

print("Nome em Upper case = " + nome.upper())
print("Nome em Lower case = " + nome.lower())
print("1ra letra com Maiscula = " + nome.capitalize())
```

Listagem 5. Exemplo51.py

No primeiro exemplo, a variável `maior_numero` receberá o valor 8.3, por ele ser o maior elemento passado por parâmetro. Já no segundo, a variável `maior_letra` receberá a letra "c", pois seu valor na tabela ASCII é superior ao das letras "a" e "b".

Outros exemplos de funções builtin são:

- ✓ **input()**, que recebe o valor que é digitado pelo usuário; e
- ✓ **float()** e **int()**, utilizadas quando se faz necessário converter valores para float e int, respectivamente.
- ✓ **max()**, retorna o valor máximo entre um intervalo
- ✓ **min()**, retorna o valor mínimo entre um intervalo
- ✓ **upper()**, coloca todas as letra em Maiúscula
- ✓ **lower()**, coloca todas as letra em minúscula
- ✓ **capitalize()**, coloca a 1ra letra em Maiúscula
- ✓ **len()**

Funções embutidas (Built-in Functions)

abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

Tabela 1: <https://docs.python.org/pt-br/3/library/functions.html>

b) Importando funções externas prontas e disponíveis

Nesta lista ([Global Module Index](https://docs.python.org/3/py-modindex.html)) você vai encontrar, em ordem alfabética, todos os módulos disponíveis na biblioteca padrão.

<https://docs.python.org/3/py-modindex.html>

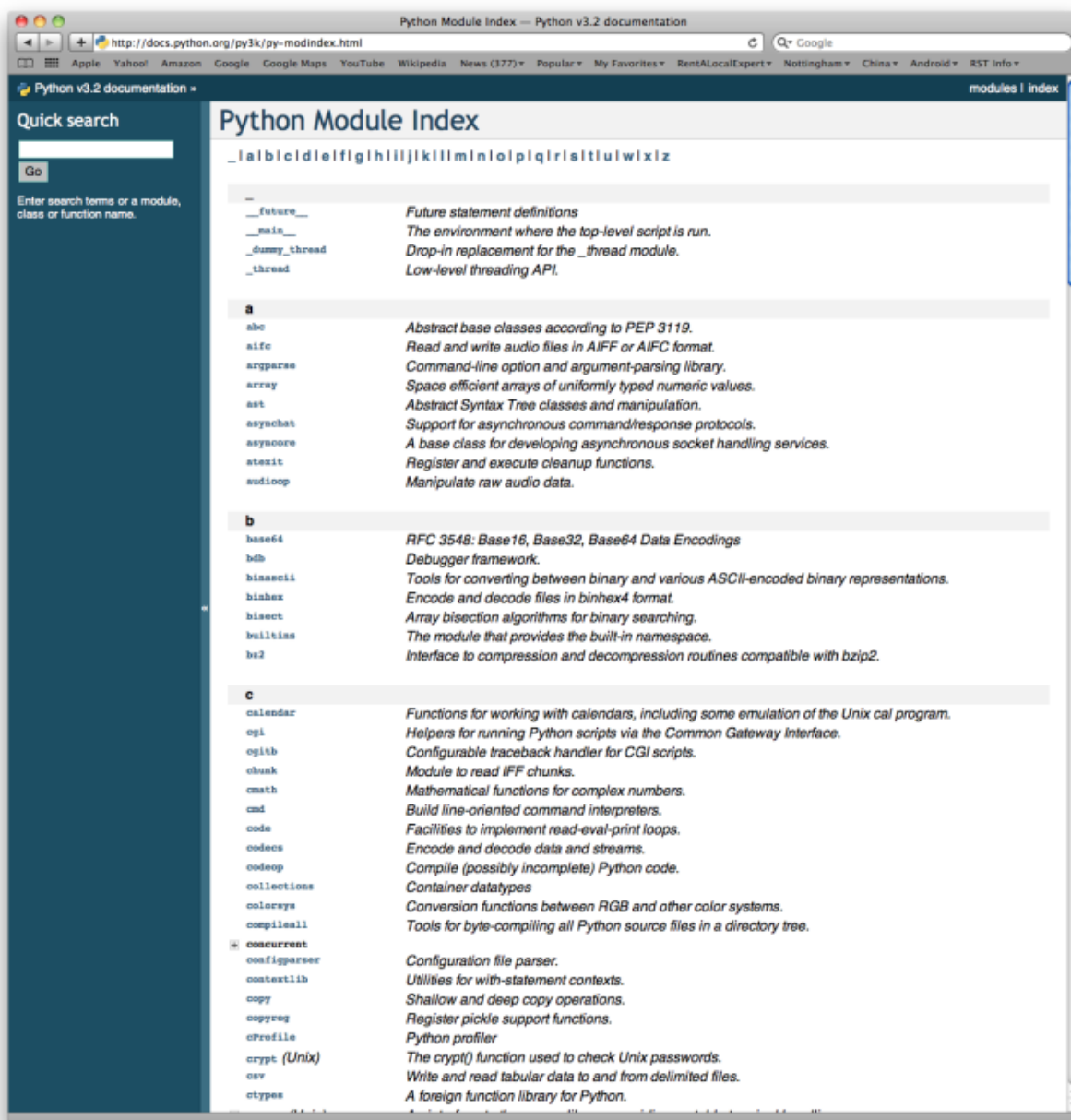


Figura 2: Global Module Index ou <https://docs.python.org/3/py-modindex.html>

O módulo math

O módulo math contém funções matemáticas que você costuma encontrar em calculadoras e algumas constantes matemáticas como ***pi*** e ***e***.

Como vimos acima, ao importar o módulo math, obtemos uma referência para um objeto *module* que contém esses elementos.

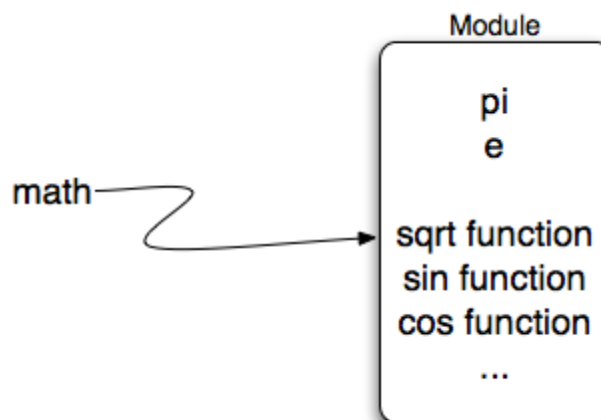


Figura 3: Função do módulo math

A seguir tem alguns itens do módulo **math**. Se você quiser saber mais detalhes, dê uma olhada na documentação do módulo em [Math Module](#).

```
import math

raiz = math.sqrt(9)
print("Funções do modulo math")
print("Valor da raiz de 9 %.2f = " %raiz)
print("Valor de Pi= ", math.pi)
PI = math.pi
print("Valor de Pi = %.4f" %PI)
print("Valor de e =",math.e)

seno= math.sin(math.radians(90))
print("Valor do senho de 90 Graus =",seno) # seno de 90 graus
```

Listagem 6: Exemplo52.py

Elas ficam todas alojadas no módulo chamado *math*. Uma vez que o módulo *math* tenha sido importado, qualquer coisa definida lá pode ser usada no nosso programa. Observe que usamos sempre o nome do módulo seguido de um *ponto* seguido pelo item específico do módulo (*math.sqrt*).

O módulo random

Muitas vezes queremos usar **números aleatórios** em programas. Aqui estão alguns usos típicos:

- Para jogar um jogo de azar em que o computador precisa jogar alguns dados, escolher um número, ou jogar uma moeda,
- Para embaralhar um baralho de cartas aleatoriamente,
- Para permitir que uma nova nave inimiga apareça aleatoriamente e atire em você,
- Para simular a precipitação de chuva em um modelo computacional para estimar o impacto ambiental da construção de uma barragem,
- Para criptografar sua sessão bancária na Internet.

O Python fornece um **módulo random** que ajuda com tarefas como esta. Você pode dar uma olhada na documentação. Aqui estão as principais coisas que podemos fazer com esse módulo.

A função **randrange()** gera um inteiro no intervalo definido pelo menor e maior argumento, usando a mesma semântica do *range* — ou seja, o valor inicial é incluído mas o final é excluído. Todos os valores têm a mesma probabilidade de ocorrência (ou seja, os resultados são *uniformemente* distribuídos).

A função **random()** retorna um número em ponto flutuante no intervalo [0.0, 1.0) – o colchete significa “intervalo fechado à esquerda” e o parênteses significa “intervalo aberto à direita”. Em outras palavras, 0.0 é possível, mas todos os números retornados serão estritamente menores que 1.0. É comum *escalar* os resultados após chamar esse método para transformá-los para dentro de um intervalo apropriado para a sua aplicação.

```
import random

print("Funções do modulo random")
print("Valor da função random.randrange(50) =",random.randrange(50))
print("Varlor da função random.randran(0, 60, 1) =",random.randrange(0, 60, 1))

prob = random.random()
print("Varlor da função random.random() =",prob)
lanceDado = random.randrange(1,7) # retorna um int, dentre 1,2,3,4,5,6
print("Valor do dado =", lanceDado)
```

Listagem 7: Exemplo53.py

Nesse caso, nós convertemos o resultado da chamada do método em um número no intervalo [0.0, 5.0). Novamente, esses números são uniformemente distribuídos — números perto do 0 são tão prováveis de aparecer quanto o 0.5, ou números perto de 1.0. Se você continuar a pressionar o botão *Run*, você verá valores aleatórios de 0.0 até o limite de, mas sem incluir o valor, 5.0.

É importante notar que geradores de números aleatórios são baseados em um algoritmo **determinístico** — reproduzível e previsível. Por isso eles são chamados de geradores **pseudo-aleatórios** — pois eles não são verdadeiramente aleatórios. Eles começam com um valor de *semente*. Cada vez que você pedir um outro número aleatório, você vai receber um com base no valor atual da semente, e o estado da semente (que é um dos atributos do gerador) é atualizado.

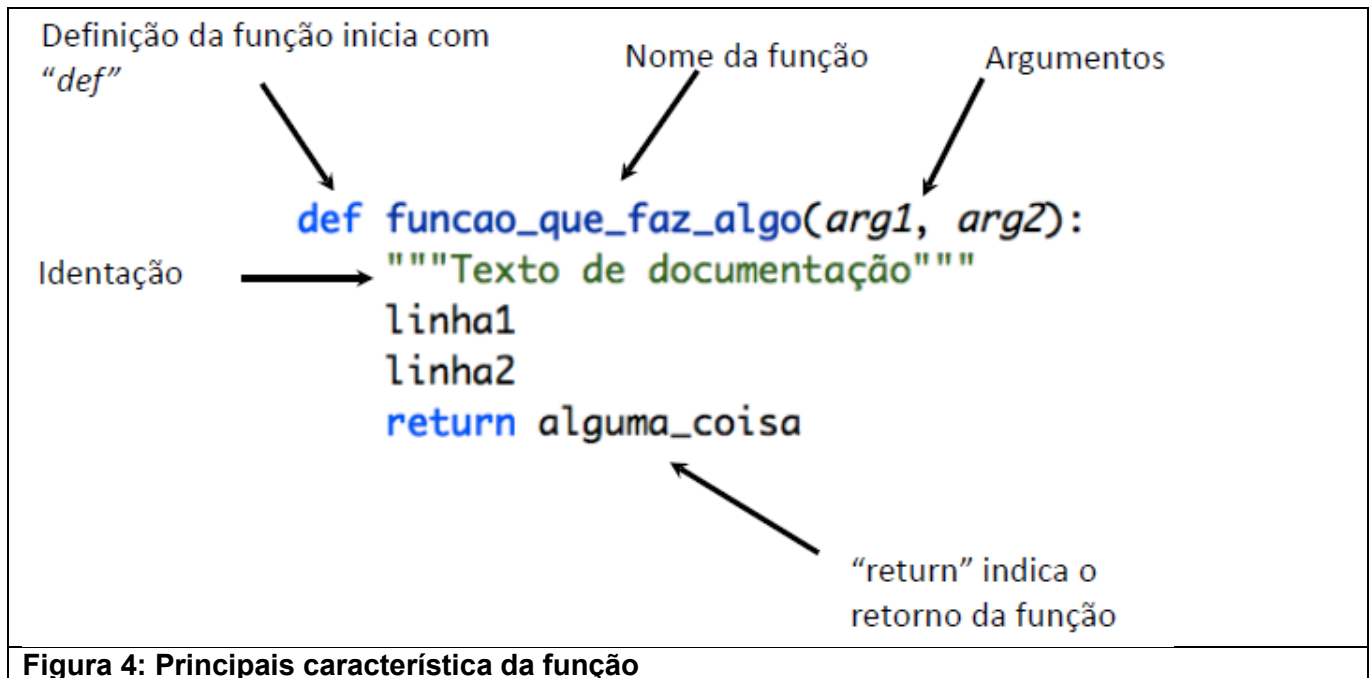
A boa notícia é que cada vez que você executar o seu programa, o valor da semente é provavelmente diferente, significando que apesar dos números aleatórios estarem sendo criados usando algoritmos determinísticos, você vai obter um comportamento aleatório cada vez que você executar.

c) Função personalizada (definida pelo programador)

Para declararmos uma função utilizamos a palavra reservada **function**. A nomenclatura das funções segue as mesmas regras de nomenclatura das variáveis.

Na hora de codificar temos 3 alternativas para definir os nomes das funções e variáveis com nomes compostos

Case	Descrição
Camel Case	Camel Case é a prática de escrever palavras compostas ou frases de modo que cada palavra ou abreviatura no meio da frase comece com uma letra maiúscula. Por exemplo ✓ dadosPessoais() ✓ “iPhone”, “eBay”.
Snake Case	Snakecase ou Underscorecase são os termos utilizados para o case que usa “_”. Snake case é bem antigo e vem do início do C, porém recebeu esse nome recentemente. Padrão na linguagem Rubi e python Exemplos ✓ dados_pessoais() ✓ calcular_soma()
Pascal Case	Pascal Case é a prática de escrever palavras compostas ou frases de modo que cada palavra ou abreviatura comece com uma letra maiúscula. Todos os nomes de membros, tipos e namespace públicos que consistem em várias palavras. ✓ DadosPessoais() ✓ “FedEx” e “HarperCollins” ✓ <i>PropertyDescriptor</i> ✓ <i>HtmlTag</i>



Funções:

- a) Criadas através de definição de funções;
- b) Todo operador chama uma função (é possível definir operadores para classes do programador, mas não é possível redefinir o operador + de inteiros, por ex.);
- c) Pode receber um ou mais argumentos;
- d) Suporta valores default (assim como C++);
- e) Argumentos não precisam estar em ordem se identificados;

```
def saudacao1(meu_nome):
    print('Olá', meu_nome)

def saudacao2(meu_nome, idade):
    print('Olá', meu_nome, '\nSua idade é:', idade)

saudacao1 ("Jão Paulo") # chama a função saudacao1 ()
saudacao2("Henrique", 9) # chama a função saudacao2 ()
```

Listagem 8: Exemplo54.py

INVOCANDO FUNÇÕES

O interpretador do Python tentará invocar uma função todas as vezes em que houver uma referência seguida por um par de parêntesis.

Ao invocarmos uma função, é necessário enviar um valor válido para cada parâmetro contido na definição. Caso não haja parâmetros, basta somente abriremos e fecharmos parêntesis.

func()

Função

- a) Toda função retorna um objeto (ou uma tupla com vários objetos);
- b) Caso não tenha valor definido pelo programador, retorna None.
- c) Uma função por si só também é um objeto.

Escopo

- a) Escopo é o nome que se dá ao conjunto de nomes acessíveis de um determinado ponto de um programa
- b) Também é chamado de **espaço de nomes** ou **namespace**
- c) Escopo são mantidos por **Namespaces**, que são mapeamentos que relacionam os nomes dos objetos aos objetos em si
- d) Quando um nome não é encontrado em nenhum ambiente de amarração a **exceção NameError** é lançada.
- e) Se o nome foi encontrado mas ainda não aconteceu nenhuma amarração a ele a exceção **UnboundLocalError** é lançada.
- f) Módulos e classes também definem escopos
- g) Ao se fazer acesso a um nome, todos os escopos, do mais interno para o mais externo, são consultados.
- h) Isto explica por que definir uma variável numa função pode fazer com que uma variável global deixe de ser acessível
- i) Um programa começa em um escopo (chamado **escopo global**) enquanto cada função acrescenta um escopo próprio (**local**)

(N.d.T: interna == *built-in*)

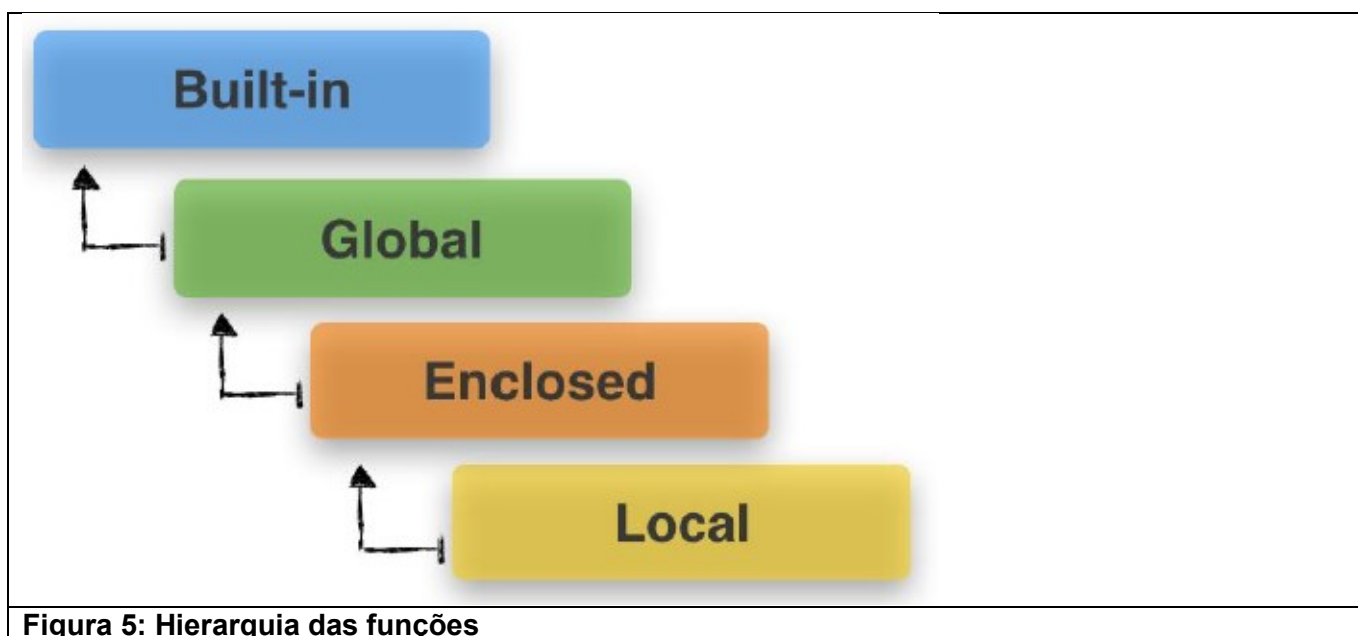


Figura 5: Hierarquia das funções

Variáveis locais e globais

- a) Variáveis definidas em funções são locais, isto é, só podem ser usadas nas funções em que foram definidas
- b) Variáveis definidas fora de funções são conhecidas como variáveis globais
- c) É possível no código de uma função ler o conteúdo de uma variável global
- d) Para alterar uma variável global, ela precisa ser declarada no corpo da função usando o comando `global`

Escopo das variáveis

Toda variável utilizada dentro de uma função tem escopo local, isto é, ela não será acessível por outras funções ou pelo programa principal.

Se houver variável com o mesmo nome fora da função, será uma outra variável, completamente independentes entre si

```
def func():
    num = 1 # variável local na função
    print("Resultado da variavel local num = ", num)

print("Exemplo de variavel local e global com funções")
num = 10 # variável global

func() # chama a função
print("Resultado da variavel global num = ", num)

print("Obs: Não é recomendavel utilizar o mesmo nome para as variavel local e global")
# Rsumindo
# Função (Variavel local) --> num = 1
# Aplicação (Variavel global) --> num = 10
```

Listagem 9: Exemplo55.py

```
def quadrado(x):
    y = x * x
    return y

aQuadrar = 10
resultado = quadrado(aQuadrar)
print("Resultado do quadrado da 10 = ", resultado)
```

Listagem 10. Exemplo56.py

Para uma variável ser compartilhada entre diversas funções e o programa principal, ela deve ser definida como **variável global**.

Para isto, utiliza-se a instrução **global** para declarara variável em todas as funções para as quais ela deva estar acessível.

O mesmo vale para o programa principal.

```
print("Exemplo de variavel local e global com funções")
num = 10 # variável global
def func():
    global num # variável local na função
    print("Resultado da variavel local num = ", num)

func() # chama a função
print("Resultado da variavel global num = ", num)

print("Caso seja necessario utilize o comando global")
```

Listagem 11. Exemplo57.py

Resultado de Funções

- Uma função tipicamente computa um ou mais valores'
- Para indicar o valor a ser devolvido como o resultado da função, usa-se o comando `return`, que tem o formato

return expressão

– onde a expressão é opcional e designa o valor a ser retornado

return -> termina a função retornando um valor

O valor default de **return** -> None

- Se a função chegar ao fim sem o uso explícito do **return**, então também será retornado o valor **None**

Funções que retornam valores são chamadas algumas vezes de **funções frutíferas**. Em muitas outras linguagens, uma função que não retorna um valor é chamado de **procedimento**.

Mas ficaremos aqui com o jeito do Python e também chamaremos isso de função, ou quando quisermos enfatizar essa propriedade de não retornar valores, chamaremos a função de **não-frutífera**.

Funções frutíferas ainda permitem ao usuário fornecer informação (argumentos). Entretanto, existe agora um dado adicional que é retornado da função.



Figura 6: Função frutífera

Como escrever funções frutíferas? Vamos começar com uma função matemática bem simples, a função quadrado.

A função quadrado terá um número como parâmetro e retornará o resultado desse número ao quadrado.

A figura 7 mostra um diagrama do código Python seguinte.

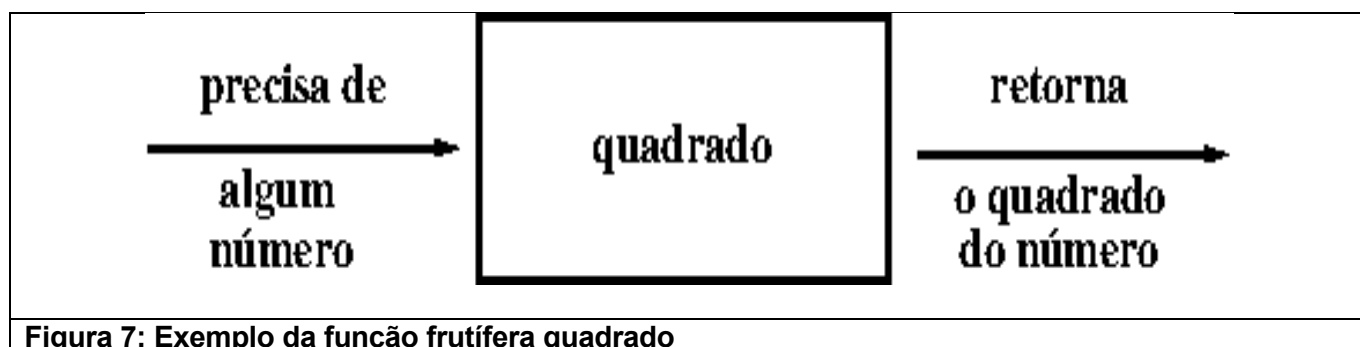


Figura 7: Exemplo da função frutífera quadrado

```
def quadrado(x):  
    y = x * x  
    return y  
  
aQuadrar = 10  
resultado = quadrado(aQuadrar)  
print("O resultado de", aQuadrar, "ao quadrado e", resultado)
```

Listagem 12: **Exemplo58.py**

O comando **return** é seguido de uma expressão que é calculada. Seu valor é retornado a quem chama a função como o *fruto* da chamada da função.

Como o comando `return` pode conter qualquer expressão, podemos evitar de criar a **variável temporária** `y` e simplesmente usar `return x*x`. Tente modificar a função `quadrado` acima para ver se funciona da mesma forma.

Por outro lado, usando **variáveis temporárias** como `y`, o programa acima permite que se encontrem erros mais facilmente. Estas variáveis temporárias são chamadas de **variáveis locais**.

Observe algo importante aqui. O nome da variável que passamos como um argumento — `aQuadrar` — não tem nada a ver com o nome do parâmetro formal — `x`. É como se `x = aQuadrar` fosse executado quando `quadrado` é chamado.

Não interessa qual o nome da variável que armazenou o valor com o qual a função é chamada. Em `quadrado`, o nome da variável é `x`. Você pode ver isto claramente no `codepens`, onde as variáveis globais e variáveis locais para a função `quadrado` aparecem em lugares separados.

Na medida em que você simula passo-a-passo o exemplo no `codepens`, note que o comando **return** não apenas causa a função retornar o valor, mas também retorna o fluxo de execução de volta ao local do programa onde a função foi chamada.

FUNÇÃO vs MÉTODO

O conceito de função e método se diferenciam só e somente só pelo retorno de valores.

Toda **função** é um bloco de instrução que, possui um nome único que a identifica em seu escopo, pode receber parâmetros e **SEMPRE retorna um valor**.

Um **método** é um bloco de instrução, possui um nome único que o identifica em seu escopo, pode receber parâmetros e **NUNCA retorna valores**.

Na prática utilizamos diversas vezes ambos os conceitos, até porque, é normal precisarmos agrupar um conjunto de instruções para que sejam executadas sequencialmente e após a execução da última instrução, o cursor volta e o programa continua de onde parou.

Outras vezes, implementaremos blocos de instruções que processarão uma informação e retornaram informações processadas, logo, estamos utilizando o conceito de funções.

No nosso caso, temos que o Python não faz distinção explícita entre ambos os conceitos. Então, o importante é somente conhecer os conceitos, até porque, na prática a implementação se tornará transparente.

PARÂMETROS vs ARGUMENTOS

Parâmetro e argumento são conceitos iguais, mas utilizados em **situações diferentes**. Para entendermos a nomenclatura temos de pensar na definição da função e no código que irá invocar a mesma.

Parâmetro	✓ é o nome utilizado quando estamos manipulando a função propriamente dita, logo, dizemos que a função declarou x parâmetros. ✓ Na implementação da função, definimos no cabeçalho, os parâmetros (variável local) que devem ser enviados quando a função for invocada.
Argumento	✓ é utilizado quando determinada função está sendo invocada, logo, dizemos que a função tem definido x argumentos. ✓ Quando estamos invocando a função, temos que definir para cada parâmetro um valor, ou seja, temos que definir os argumentos que a função irá levar consigo.

O Python trata os parâmetros de funções como uma **tupla** ou **dicionário**, inclusive, podemos definir os valores que uma função receberá e no final deixar uma vírgula sobrando, da mesma forma em que podemos fazer quando estamos definindo os valores numa tupla.

```
def func(param1, param2, param3):
    pass
```

Figura 8: definição de uma função com seus respectivos parâmetros

Assim, temos que parâmetro é a variável local que foi definida no cabeçalho da função e que será utilizada no bloco de instrução dela, enquanto argumento, é o valor que será passado ao invocarmos a funções.

Uma função deve receber os valores na ordem em que os parâmetros estão definidos, do contrário, ocorrerá os mais diversos problemas ou então, o Script nem será executado.

PARÂMETROS DEFAULT EM PYTHON

Um parâmetro default é um parâmetro com um valor pré-definido, se o usuário não passar nenhum valor para um parâmetro default ele assume o valor pré definido.

Os argumentos default sempre devem aparecer depois que todos os argumentos posicionais (aqueles que dependem só da posição).

Por exemplo, vamos implementar uma simples função e definir um único parâmetro default.

```
print("Exemplos da função Default")
def funcao_default(param1=1000):
    print("Resultado da 1ra função default = ", param1)

funcao_default() ## Chamada da função default funcao_default()
```

Listagem 13: **Exemplo59.py**

Como pode ser visto no código acima, implementamos uma função de nome **funcao_default ()** e definimos no cabeçalho da mesma [entre parêntesis] um parâmetro de nome param1 e associamos a este nome o valor inteiro 1000.

```
print()
print("2da função default")

def login(sistema="Ubuntu", usuario="root", senha="123"):
    print("Valor da variavel Sistema = ", sistema)
    print("Valor da variavel Usuário = ", usuario)
    print("Valor da variavel da Senha = ", senha)

login() # Chamada da função default login()
```

Listagem 14: **Exemplo59.py**

A função **login()**, definida no código acima, pode ser invocada sem que seja passado um único argumento, até porque, por padrão, há valores definidos para todos os parâmetros declarados.

É importante dizer que, se invocarmos uma função sem passarmos os parâmetros que estão definidos, o Python levantará uma exceção, porém, com o uso de parâmetros default, a obrigatoriedade torna-se facultativo.

```
def dados_pessoais(nome, sobrenome, idade, sexo):

    print("Nome: {}\nSobrenome: {}\nIdade: {}\nSexo: {}".format(nome, sobrenome, idade, sexo))

dados_pessoais(nome="Henrique", sobrenome="Gustavo", idade=20, sexo="Masculino")
```

Listagem 15: **Exemplo59.py**

ARGUMENTO DE FUNÇÃO NOMEADO VS POSICIONAL EM PYTHON

ARGUMENTOS NOMEADOS (KEYWORD ARGUMENTS)

Em Python é possível invocarmos funções passando os argumentos de forma posicional ou então, através da associação chave-valor.

Argumento	Descrição
Posicional	✓ é o nome utilizado para a passagem de valores onde cada valor estará na ordem conforme implementado na função.
Nomeado	✓ é a passagem de valores fazendo associação com o nome do parâmetro e o valor que está sendo enviado.

Argumento nomeado é um recurso que permite a passagem de valores pela associação chave-valor e assim, eles não precisam estar dispostos numa ordem pre-estabelecida, até porque, o Python receberá um dicionário que contém o nome e o valor de cada parâmetro.

Se analisarmos a forma que passamos valores e as estruturas estudadas, é possível perceber que a passagem de

- parâmetros posicionais**, nada mais é do que o envio de uma tupla, enquanto que
- a **passagem de valores nomeados**, é o envio de um dicionário

A única regra em relação a passagem de valores através de parâmetros nomeados e parâmetros posicionais, é que devemos escolher uma das formas de envio, isto é, ou utilizamos parâmetros posicionais ou então, parâmetros nomeados.

Função de soma definida e através de lambda

Funções lambda (funções anônimas)

Funções lambda são funções anônimas de uma única linha que aceitam argumentos.

Script	Saída
<pre> lambFun = lambda x1, x2: x1 + x2 print(lambFun(1,2)) print(lambFun(1,2.3)) </pre>	<pre> 3 3.3 </pre>

Figura 8: Exemplo de função lambda

Dada a demanda do público, algumas características encontradas em linguagens de programação funcionais (como Lisp) foram adicionadas a Python. Com a palavra-chave lambda, funções curtas e anônimas podem ser criadas.

Aqui está uma função que devolve a soma de seus dois argumentos: 'lambda a, b: a+b'. Funções Lambda podem ser utilizadas em qualquer lugar que exigiria uma função tradicional. Sintaticamente, funções Lambda estão restritas a uma única expressão.

Semanticamente, ela são apenas açúcar sintático para a definição de funções normais. Assim como definições de funções aninhadas, funções lambda não podem referenciar variáveis de um escopo mais externo:

Para os seguintes enunciados utilize a função lambda:

- Elabore um programa que converte a temperatura de Fahrenheit para Celsius:
- Faça um programa que solicite o nome do usuário e a idade do usuário, depois disso exiba a mensagem: "{nome} possui {idade} anos.". Esta mensagem deve ser escrita em uma função lambda.
- Faça um programa que solicite dois números ao usuário e exiba a multiplicação deles. A multiplicação deve ser calculada em uma função lambda.

```

# Programa para converter temperatura de Fahrenheit para Celsius
print("Conversor de temperatura de Fahrenheit para Celsius ")
temp_celsius = lambda f: (5/9) * (f - 32)

f = float(input('Entre com a temperatura em Fahrenheit: '))
print('A temperatura em Celsius é: {0:.2f}'.format(temp_celsius(f)))
print()

# Programa que calcula a área de um retângulo e triângulo

retangulo = lambda lado_a, lado_b: lado_a * lado_b
triangulo = lambda lado, altura: (lado * altura) / 2
circulo = lambda raio: 3.14 * (raio ** 2)

print("Digite os lados do retangulo")

```

```
lado1 = float(input("Digite o lado 1 = "))
lado2 = float(input("Digite o lado 2 = "))
print("Area do retangulo = ", retangulo(lado1, lado2))
print()

print("Digite os valores do triangulo")
base = float(input("Digite o valor da base = "))
altura = float(input("Digite o valor da altura = "))
print("Area do triangulo = ", triangulo(base, altura))

escrever_nome_idade = lambda nome, idade: print(nome, "possui", idade, "anos.")
nome_digitado = str(input("Digite seu nome: "))
idade_digitada = int(input("Digite sua idade: "))

escrever_nome_idade(nome_digitado, idade_digitada)

multiplicar = lambda numero1, numero2: numero1 * numero2

numero = int(input("Digite o primeiro número: "))
numero2 = int(input("Digite o segundo número: "))
resultado = multiplicar(numero, numero2)
print("Resultado da Multiplicação = ", resultado)
```

Listagem

Glossário

argumento

Um valor fornecido para uma função quando a função é chamada. Este valor é atribuído ao parâmetro correspondente na função. O argumento pode ser o resultado de uma expressão que pode envolver operadores, operandos e chamadas a outras funções frutíferas

biblioteca padrão

Uma coleção de módulos que faz parte de uma instalação padrão do Python.

chamada de função

Um comando que executa uma função. Ele consiste do nome da função seguido por uma lista de argumentos entre parênteses.

comando composto

Um comando que consiste de duas partes:

1. cabeçalho - que começa com uma palavra reservada determinando o tipo do comando e que termina com dois pontos (:).
2. corpo - contendo um ou mais comando tabulados igualmente a partir do cabeçalho.

A sintaxe de um comando composto se assemelha com:

```
palavra-reservada expressão:
    comando
    comando ...
```

composição de funções

Usando a saída de uma função como entrada de outra função.

corpo

A segunda parte de um comando composto. O corpo consiste de uma sequência de comandos todos igualmente tabulados a partir do começo do cabeçalho. A quantidade padrão de tabulação usada pela comunidade do Python é 4 espaços.

determinístico

Um processo que é reproduzível e previsível.

documentação

Um lugar onde você pode pegar informação detalhada sobre aspectos da linguagem de programação.

declaração import

uma declaração que permite funções e variáveis definidas em outro módulo de Python serem trazidas ao ambiente de outro script. Para usar as construções da tartaruga, precisamos primeiro importar o módulo turtle.

definição de função

Uma declaração que cria uma nova função, especificando o seu nome, parâmetros e os comandos que ela executa.

duração

Variáveis e objetos tem duração — eles são criado em algum ponto durante a execução do programa e são destruídos em algum outro ponto.

docstring

Se a primeira coisa no corpo da função (ou, como veremos mais tarde, em outros lugares também) é um string, ele é anexado à função como seu atributo `__doc__`.

fluxo de execução

A ordem em que comandos são executados durante a execução de um programa.

função

Uma sequência de comandos com um nome que executa operações úteis. Funções podem ou não ter parâmetros e podem ou não produzir um resultado.

função frutífera

Uma função que retorna um valor quando ela é chamada.

gerador de números aleatórios

Uma função que te fornece números aleatórios, geralmente entre 0.0 e 1.0.

linha de cabeçalho

A primeira parte de um comando composto. O cabeçalho começa com uma palavra reservada a termina com dois pontos (:).

módulo

Um arquivo contendo definições e comandos em Python para serem usados em outros programas em Python. Os conteúdos de um módulo são disponibilizados para outro programa pelo uso do comando *import*.

número aleatório

Um número que é gerado de forma a exibir aleatoriedade estatística.

número pseudo-aleatório

Um número que não é genuinamente aleatório mas criado a partir de um algoritmo.

parâmetro

Um nome usado dentro de uma função para se referir ao valor que foi passado a ela como um argumento.

refatorar

Processo de reorganizar o código de seu programa, usualmente para torná-lo mais fácil de entender. Tipicamente, temos um programa que já está funcionando, então voltamos para fazer uma

“arrumação”. Em geral envolve escolher nomes melhores para as variáveis ou perceber um padrão repetido e mover esse código para uma função.

variável local

Uma variável definida dentro de uma função. Uma variável local pode somente ser usada em sua função. Parâmetros de uma função são também um tipo especial de variável local.

Referência Bibliográfica

- ✓ Python, Escreva seus primeiros programas, Felipe Cruz, Casa do Código.
- ✓ Curso Introdutório de Python, Grupy-Sanca, 09 de maio de 2019
- ✓ Tutorial Python, Release 2.4.2. Guido van Rossum, Fred L. Drake, Jr., editor. Tradução: Python Brasil
- ✓ Curso Python 3. Juracy Filho e Leonardo Leitão. Cod3r.com.br
- ✓ https://aprendacompy.readthedocs.io/pt/latest/capitulo_01.html
- ✓ <https://www.devmedia.com.br/python-trabalhando-com-variaveis/38644>
- ✓ <https://cadernodelaboratorio.com.br/2017/12/01/variaveis-em-python3/>
- ✓ <https://cadernoscicomp.com.br/tutorial/introducao-a-programacao-em-python-3/funcoes/>

LINKS EXTERNOS

- [Function Definition](#)
- [Python Data Model - The standard type hierarchy](#)
- [Calls](#)
- [Argument](#)

Como depurar o Python no VS Code

Python for VSCode tht13.python Thomas Haakon Townsend1.514.338Repository Python language extension for vscode
Python ms-python.python Microsoft18.134.428RepositoryLicense Linting, Debugging (multi-threaded, remote), Intellisense, Jupyter Notebooks, code formatting, refactoring, unit tests, snippets, and more. Reload Required Please reload Visual Studio Code to enable the updated extension.
MagicPythonmagicstack.magicpython MagicStack Inc.1.319.863RepositoryLicense Syntax highlighter for cutting edge Python. • Disable • Uninstall • This extension is enabled globally

Para aparecer os comandos digitar:

Shift + Control + P

Parte 2 – Prática (mão na massa)!!!

Para as atividades práticas vamos utilizar o VS Code, que deve estar previamente configurado para executar o python)

Para a criação dos arquivos (com extensão .py) vamos utilizar o editor de texto

Para executar, deve utilizar o interpretador do VS Code, temos 2 opções

- Apertar o Botão direito e selecionar a opção **RUN PYTHON TERMINAL**
- No menu superior, selecionar Debug -> **START WITHOUT DEBUGGING**

Para limpar o console, utilizar o comando clear ou cls

2) Criação do Projeto no VS CODE

- Para isto crie uma pasta no seu pendrive com o nome **IPE-Python-2023**.
- Dentro do **IPE-Python-2023**, criar a pasta **Aula05-Python**
- Utilizando o VS CODE acesse a pasta criada (**IPE-Python-2023/ Aula05-Python**)
- Para isso, acione a opção **File > New > File** e crie os seguintes (exemplo50.py, exemplo51.py exemplo59.py), para melhor organização (**BOA PRÁTICA!!!**)

Exemplo 50

PASTA: IPE-Python-2023/ Aula05-Python

Arquivo: exemplo50.py

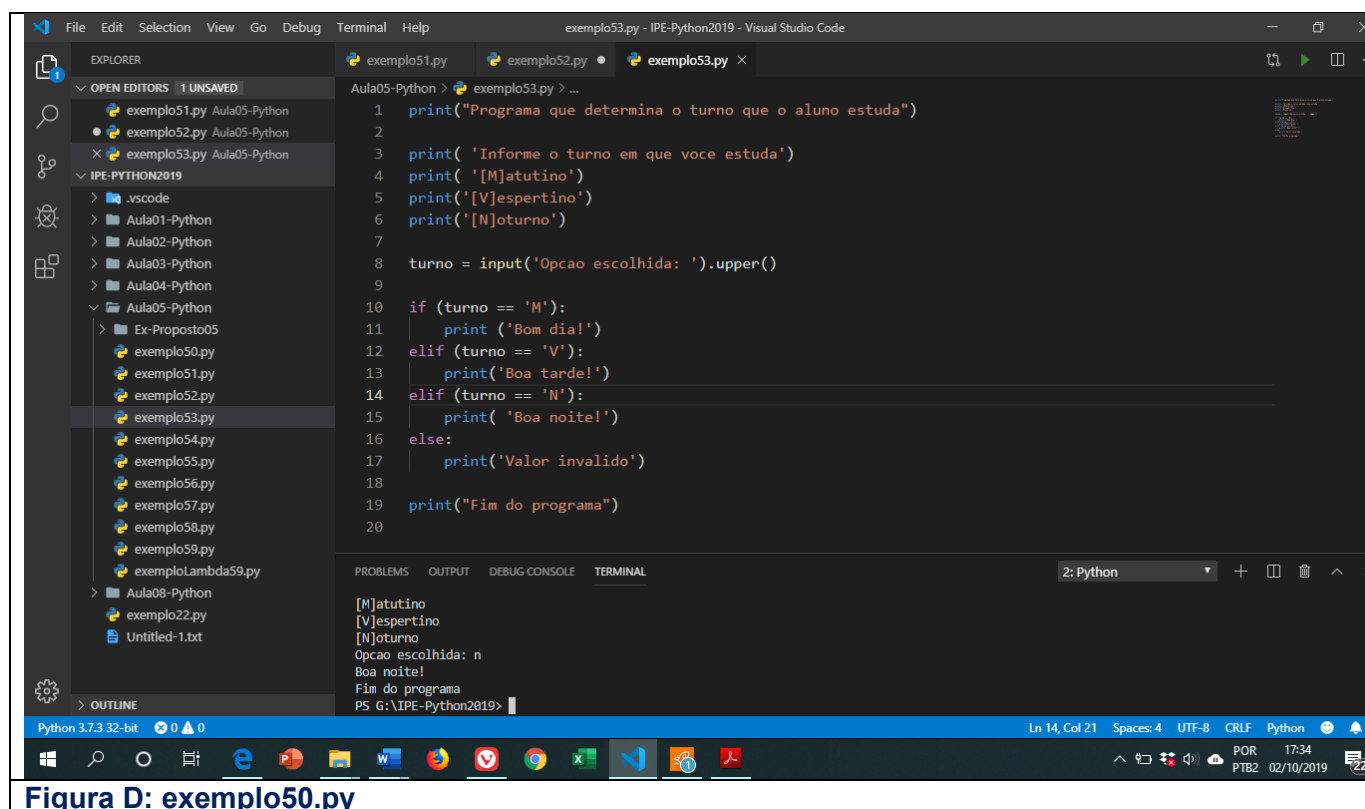


Figura D: exemplo50.py

Exemplo 51**PASTA: IPE-Python-2023/ Aula05-Python****Arquivo: exemplo51.py**

```

1 print("Funções internas - builtin ou embutidas")
2 maior_numero = max(3 * 11, 5**3, 512 - 9, 1024**0)
3 menor_numero = min(3, 6, 7, 10)
4 maior_letra = max('a', 'b', 'c')
5 menor_letra = min('a', 'b', 'c')
6
7 print("O Nro maximo da lista =", maior_numero)
8 print("O Nro minimo da lista =", menor_numero)
9 print("A maior letra da lista =", maior_letra)
10 print("A menor letra da lista =", menor_letra)
11
12 nome = "henrique"
13 nome.lower()
14 nome.upper()
15 nome.capitalize()
16
17 print("Nome em Upper case = " + nome.upper())
18 print("Nome em Lower case = " + nome.lower())
19 print("1ra letra com Maiuscula = " + nome.capitalize())
20
21

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

2: Python

O Nro minimo da lista = 3
A maior letra da lista = c
A menor letra da lista = a
Nome em Upper case = HENRIQUE
Nome em Lower case = henrique
1ra letra com Maiuscula = Henrique
PS G:\IPE-Python2019> []

Figura E: exemplo51.py**Exemplo 52****PASTA: IPE-Python-2023/ Aula05-Python****Arquivo: exemplo52.py**

```

1 import math
2 import random
3
4 print("Funções do modulo math")
5 print("Potencia da base 2 e elevado a 3 = ", math.pow(2, 3))
6 print("Potencia da base 7 e elevado a 7 = ",math.pow(7, 4))
7
8 print("Valor absoluto de 5 = ",abs(5))
9 print("Valor absoluto de -5 = ", abs(-5))
10
11 raiz = math.sqrt(9)
12 print()
13 print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
14 Prints the values to a stream, or to sys.stdout by default. Optional keyword arguments: file: a
15 file-like object (stream); defaults to the current sys.stdout. sep: string inserted between values,
16 default a space. end: string appended after the last value, default a newline. flush: whether to
17 forcibly flush the stream.
18 print("Valor do seno de 90 Graus =", seno) # seno de 90 graus
19
20
21 print()
22 print("Funções do modulo random")
23 print("Valor da função random.randran(50) =",random.randrange(50))
24 print("Valor da função random.randrange(0, 50, 1) =", random.randrange(0, 50, 1))

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

2: Python

Funções do modulo random
Valor da função random.randrange(50) = 4

Figura F: exemplo52.py

Exemplo 53**PASTA: IPE-Python-2023/ Aula05-Python****Arquivo: exemplo53.py**

```

1 def saudacao1(meu_nome):
2     print('Olá', meu_nome)
3
4
5 def saudacao2(meu_nome, idade):
6     print('Olá', meu_nome, '\nSua idade é:', idade)
7
8
9 saudacao1("João Paulo") # chama a função saudacao1 ()
10 saudacao2("Henrique", 9) # chama a função saudacao2 ()
11
12
13

```

PS G:\IPE-Python2019> & C:\Users\SERGIOMEDINA\AppData\Local\Programs\Python\Python37-32\python.exe g:/IPE-Python2019/Aula05-Python/exemplo53.py
Olá João Paulo
Olá Henrique
Sua idade é: 9
PS G:\IPE-Python2019>

Figura G: exemplo53.py**Exemplo 54****PASTA: IPE-Python-2023/ Aula05-Python****Arquivo: exemplo54.py**

```

1 print("Exemplo de variavel local e global com funções")
2 num = 10 # variável global
3 def func():
4     num = 1 # variável local na função
5     print("Resultado da variavel local num = ", num)
6
7 func() # chama a função
8 print("Resultado da variavel global num = ", num)
9
10 print("Obs: Não é recomendavel utilizar o mesmo nome para as variavel local e global")
11 # Rsumindo
12 # Função (Variavel local) --> num = 1
13 # Aplicação (Variavel global) --> num = 10
14
15 def quadrado(x):
16     y = x * x
17     return y
18
19 aQuadrar = 10
20 resultado = quadrado(aQuadrar)
21 print("Resultado do quadrado da 10 = ", resultado)
22
23

```

Sua idade é: 9
PS G:\IPE-Python2019>

Figura H: exemplo54.py

Exemplo 55**PASTA: IPE-Python-2023/ Aula05-Python****Arquivo: exemplo55.py**

```

1 def verificar_senha(senha):
2     if (senha == 123456):
3         print("Bemvindo ao sistema")
4         print("Vc digitou =", senha_digitada)
5     else:
6         print("Senha não confere")
7         print("Vc digitou =", senha_digitada)
8
9
10 """
11 senha_digitada = input("Digite a sua senha = ")
12 verificar_senha(senha_digitada)
13 """
14
15 from getpass import getpass
16 senha_digitada = getpass("Digite a sua senha = ")
17 verificar_senha(senha_digitada)
18
19
20
21

```

Terminal output: PS C:\Users\SERGIO\MEDINA\Google Drive\Lab-IPE-Python2023>

Figura I: exemplo55.py**Exemplo 56****PASTA: IPE-Python-2023/ Aula05-Python****Arquivo: exemplo56.py**

```

1 print("Exemplos da função Default")
2 def funcao_default(param1=1000):
3     print("Resultado da 1ra função default = ", param1)
4
5 funcao_default() # Chamada da função default funcao_default()
6 print()
7 print("2da função default")
8 def login(sistema="Ubuntu", usuario="root", senha="123"):
9     print("Valor da variavel Sistema = ", sistema)
10    print("Valor da variavel Usuário = ", usuario)
11    print("Valor da variavel da Senha = ", senha)
12
13 login() # Chamada da função default login()
14
15 print()
16 def dados_pessoais(nome, sobrenome, idade, sexo):
17     print("Nome: {}\nSobrenome: {}\nIdade: {}\nSexo: {}".format(
18         nome, sobrenome, idade, sexo))
19
20 dados_pessoais(nome="Henrique", sobrenome="Gustavo", idade=20, sexo="Masculino")
21

```

Terminal output:
Nome: Henrique
Sobrenome: Gustavo
Idade: 20
Sexo: Masculino
PS G:\IPE-Python2019>

Figura J: exemplo56.py

Exemplo 57**PASTA: IPE-Python-2023/ Aula05-Python****Arquivo: exemplo57.py**

```

1  def entrada():
2      peso_dig = float(input("Digite o peso = "))
3      altura_dig = float(input("Digite a altura = "))
4      return (peso_dig, altura_dig)
5
6
7  def calcula_imc(peso, altura):
8      print(peso / (altura ** 2))
9      imc = peso / (altura ** 2)
10     return (imc)
11
12 p,a = entrada()
13 print("Calculo do IMC =", calcula_imc(p, a))
14

```

TERMINAL

```

Calculo do IMC = 24.535123966942148
PS C:\Users\SERGIOMEDINA\Google Drive\Lab-IPE-Python2023>

```

Figura k: exemplo57.py**Exemplo 58****PASTA: IPE-Python-2023/ Aula05-Python****Arquivo: exemplo58.py
modulo1.py**

```

1  import modulo1 as mod1
2
3
4  p,a = mod1.entrada()
5  print("Calculo do IMC =", mod1.calcula_imc(p, a))
6

```

TERMINAL

```

PS C:\Users\SERGIOMEDINA\Google Drive\Lab-IPE-Python2023> & C:\Users\SERGIOMEDINA\AppData\Local\Programs\Python\Python310\python.exe
"c:\Users\SERGIOMEDINA\Google Drive\Lab-IPE-Python2023\Aula05-Função_Python\exe_Funcao58.py"
Digite o peso = 76
Digite a altura = 1.76
24.535123966942148
Calculo do IMC = 24.535123966942148
PS C:\Users\SERGIOMEDINA\Google Drive\Lab-IPE-Python2023>

```

Figura l: exemplo58.py

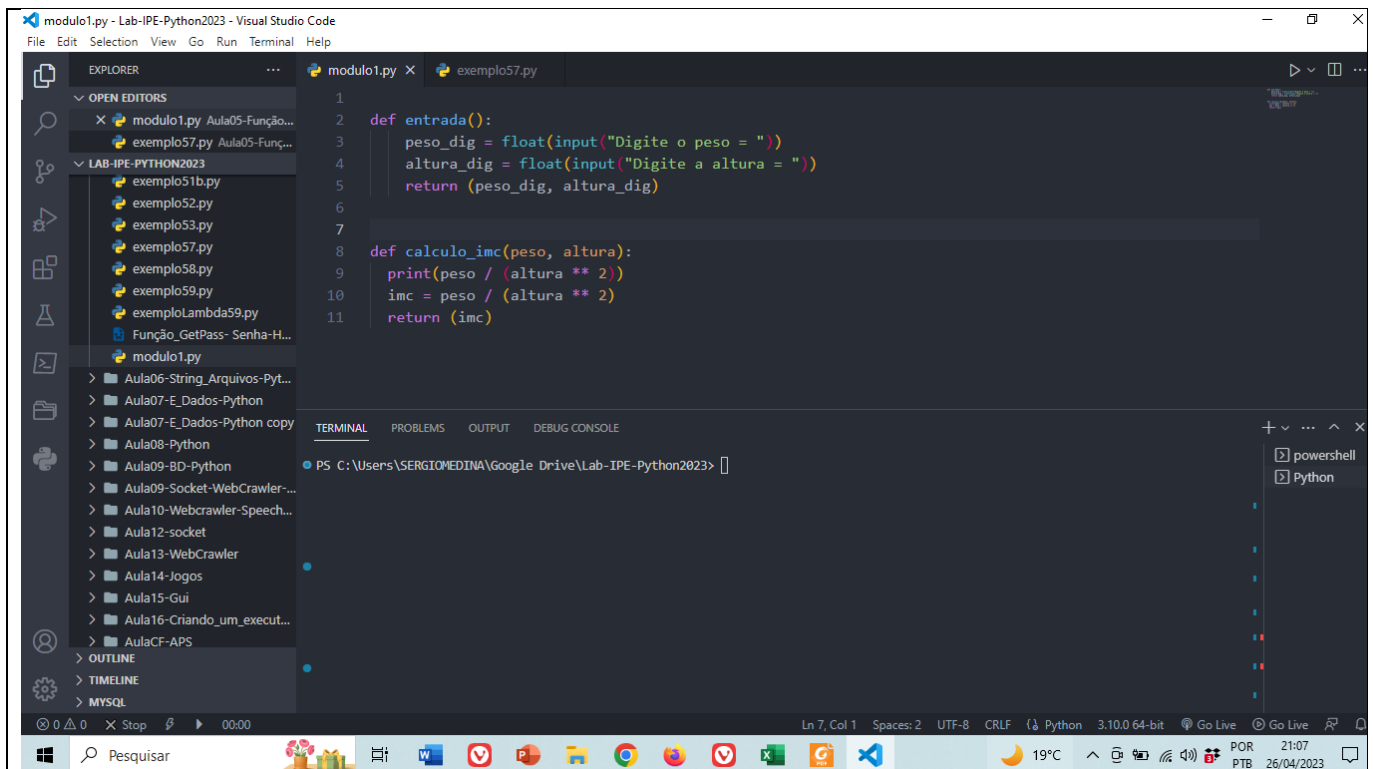


Figura m: modulo1.py

Exemplo 59

PASTA: IPE-Python-2023/ Aula05-Python

Arquivo: exemploLambda59.py

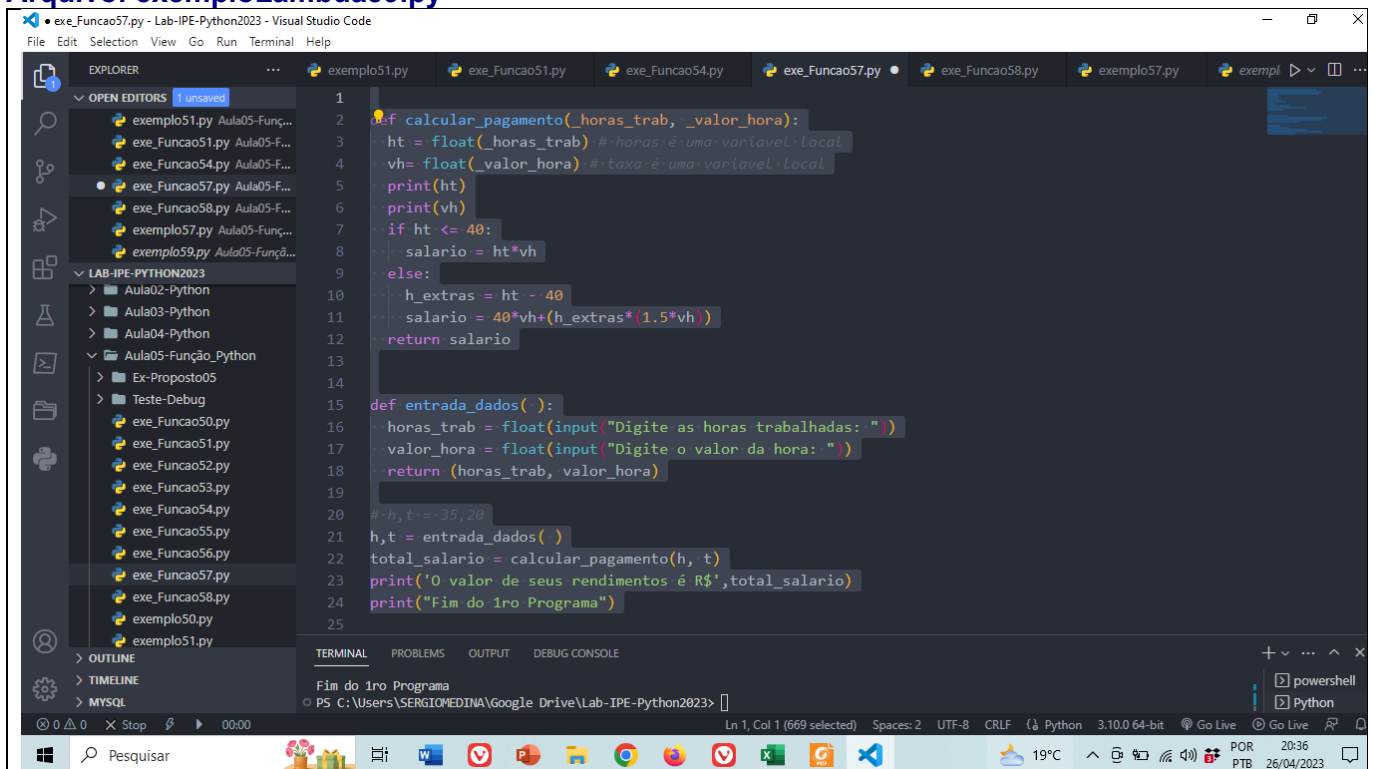
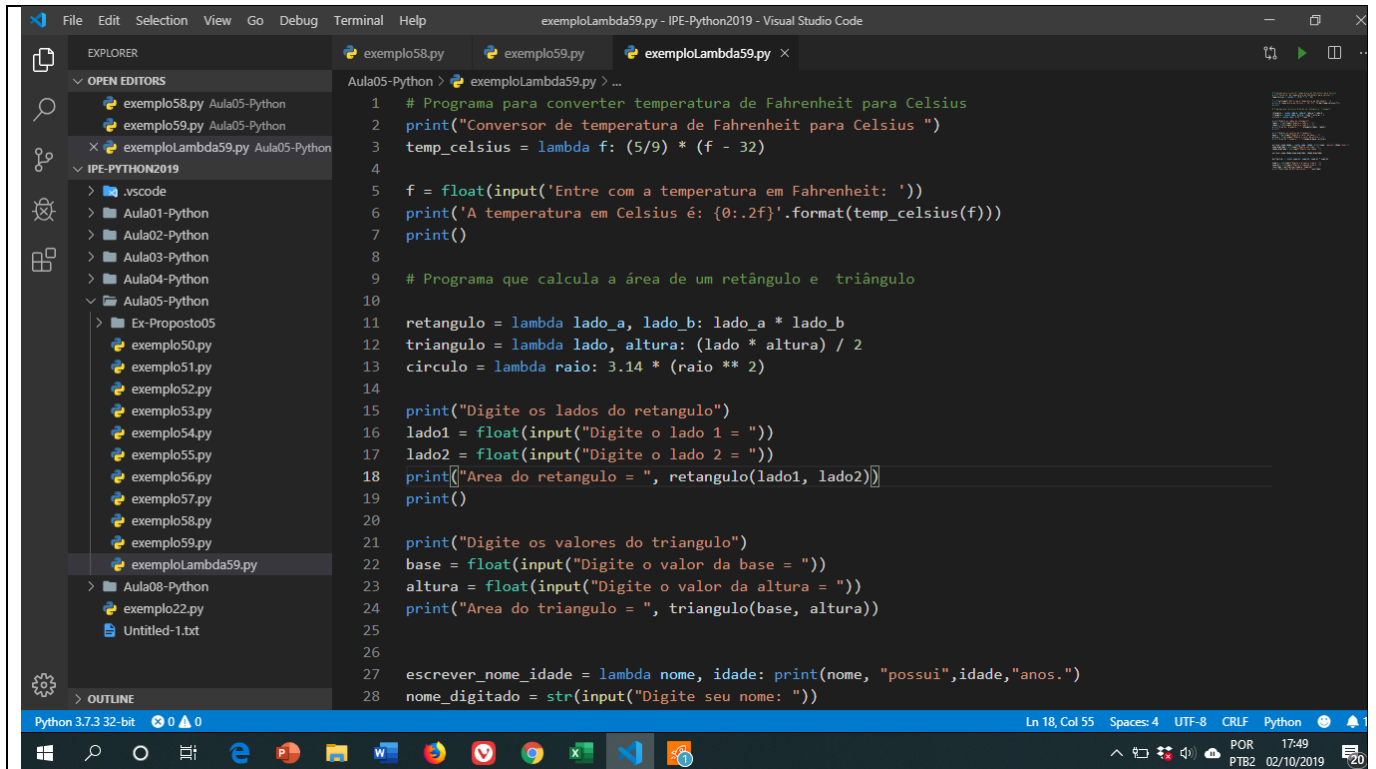


Figura k: exemplo59.py

Exemplo Lambda 58

PASTA: IPE-Python-2023/ Aula05-Python

Arquivo: exemploLambda58.py



```
1 # Programa para converter temperatura de Fahrenheit para Celsius
2 print("Conversor de temperatura de Fahrenheit para Celsius ")
3 temp_celsius = lambda f: (5/9) * (f - 32)
4
5 f = float(input('Entre com a temperatura em Fahrenheit: '))
6 print('A temperatura em Celsius é: {:.2f}'.format(temp_celsius(f)))
7 print()
8
9 # Programa que calcula a área de um retângulo e triângulo
10
11 retangulo = lambda lado_a, lado_b: lado_a * lado_b
12 triangulo = lambda lado, altura: (lado * altura) / 2
13 circulo = lambda raio: 3.14 * (raio ** 2)
14
15 print("Digite os lados do retangulo")
16 lado1 = float(input("Digite o lado 1 = "))
17 lado2 = float(input("Digite o lado 2 = "))
18 print("Area do retangulo = ", retangulo(lado1, lado2))
19 print()
20
21 print("Digite os valores do triangulo")
22 base = float(input("Digite o valor da base = "))
23 altura = float(input("Digite o valor da altura = "))
24 print("Area do triangulo = ", triangulo(base, altura))
25
26
27 escrever_nome_idade = lambda nome, idade: print(nome, "possui", idade, "anos.")
28 nome_digitado = str(input("Digite seu nome: "))
```

Figura J: exemploLambda57.py